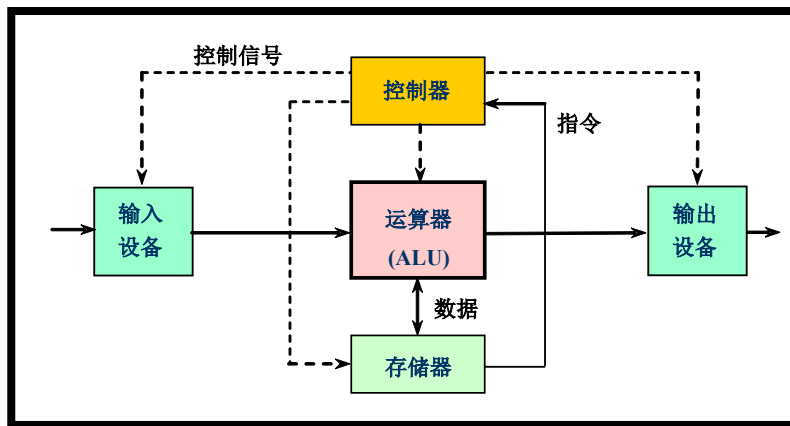


计组复习背诵提纲

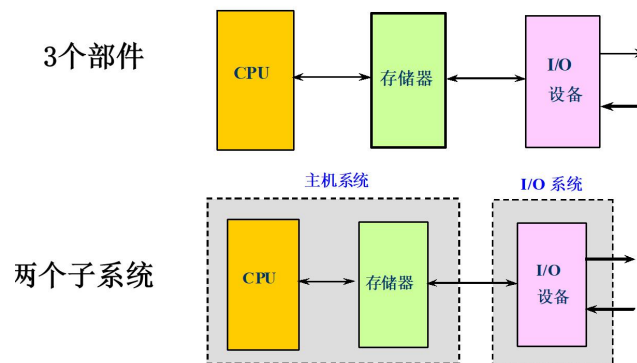
（注：本材料由不是很懂计组的人在期末补天的短暂时间内整理而成，因此必然存在大量错误，请谅解。本材料结合了 PPT、课本和一些复习材料整理而成，包含了几乎必考的大题题型、张老师期末画的课后概念题和需要背诵记忆的图，如题“背诵提纲”，没有特殊标注的地方恐怕都是要背诵记忆的，加油！）

第 1 章 计算机系统概论

1. 冯·诺依曼结构匈牙利数学家冯·诺依曼 1946 年 提出



2. 硬件结构



CPU = 运算器 + 控制器

存储器 = 内存（主存）+ 外存（辅存）

主机 = CPU + 主存储器

I/O 设备 = 输入设备 + 输出设备

3. 名词解释

运算器：计算机的执行部件，用于对数据的加工处理，完成算术运算和逻辑运算

控制器：计算机的管理机构和指挥中心，它协调计算机的各个部件自动地工作。

存储器：计算机的存储部件，存储程序和数据。

内存：按地址访问的一维线性空间

输入输出设备：计算机与外界联系的重要桥梁，是计算机系统中的一个不可或缺的组成部分

输入设备：作用：将程序和数据以计算机所能识别的形式输入到计算机内

输出设备：作用：将计算机处理的结果以人们所能接受或其它系统所要求的形式输出到外部世界。

操作系统：负责管理系统资源，为应用程序提供运行环境，为用户提供操作界面。

MIPS:百万条指令/秒

MFLOPS:百万次浮点运算/秒

4. 计算机的性能指标（必背，常考）

主频、运算速度、基本字长、主存容量和主存存取周期

第 6 章 计算机执行程序的过程（背）

三个部件分别是：运算部件、控制器和内存

1. 第一条指令的执行过程（记住这条就基本全部记住了）

指令：Load R1, 200 (R0)

功能：MEM[[R0]+200]→R1

即用[R0]+200 作为地址访问存储器，将读出的内容送给 R1。

第 1 步：取指令

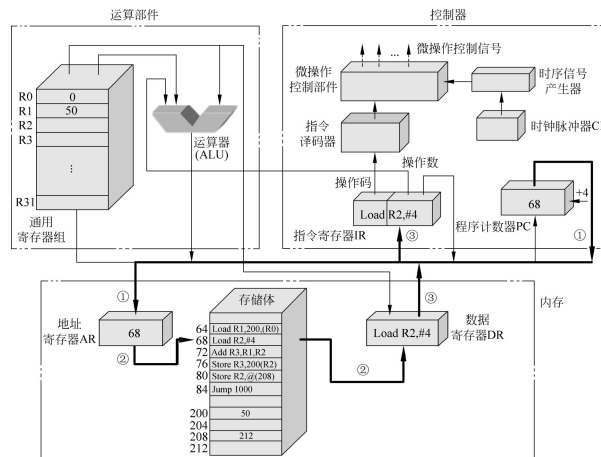
[PC]→AR。

将 PC 的内容（64）传送到内存的地址寄存器 AR。如图 1 中的粗线所示。

从存储器读出第一条指令，放到数据寄存器 DR。如图 2 中的粗线所示。

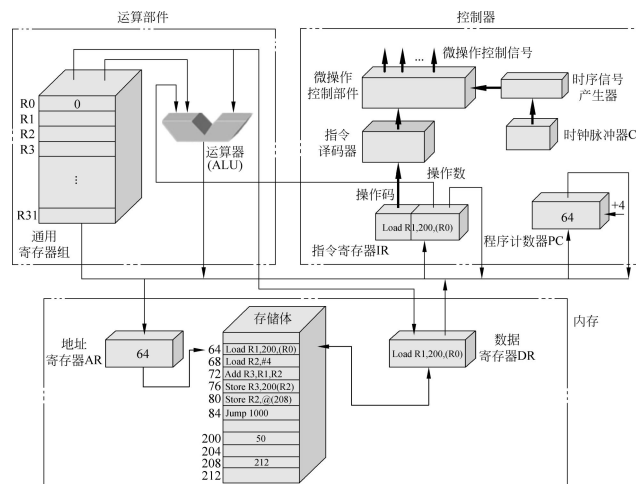
把该指令从 DR 送到指令寄存器 IR。如图 3 中的粗线所示。

（这张图用的是第 2 条指令的，看箭头就行了，不用管 PC 和 AR,DR 中的内容）



第2步：译码

指令译码器进行译码，控制器结合时钟脉冲信号 CP 和时序信号产生器所生成的节拍信号，产生执行该指令所需要的微操作控制信号。



第3步：执行指令

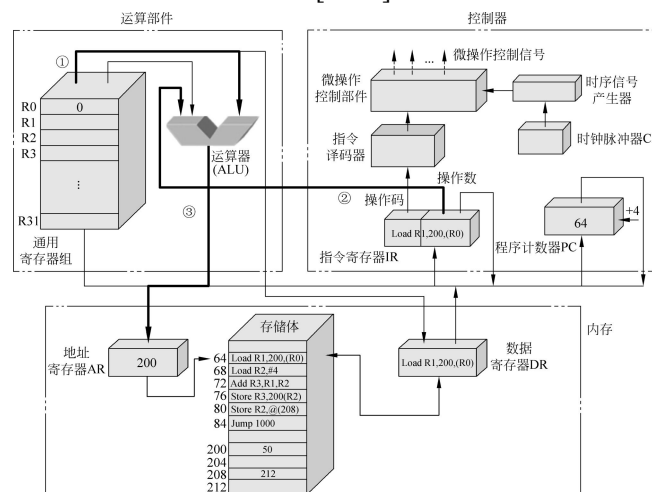
计算访存地址：[R0]+200→AR

具体操作：

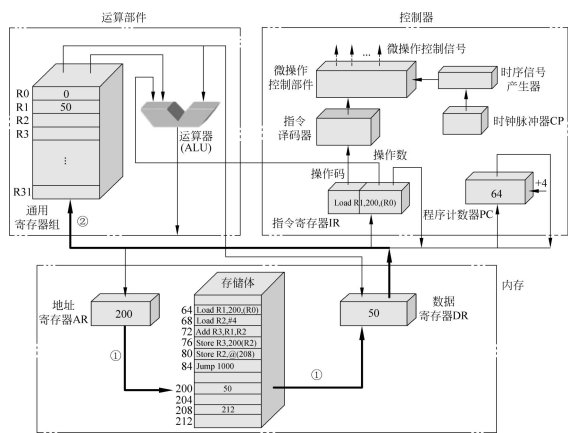
[R0]→ALU // 把寄存器 R0 中的内容（为 0）送给 ALU。如图 5 中的①所示。

IR 中的操作数“200”→ALU。如图 6 中的②所示。

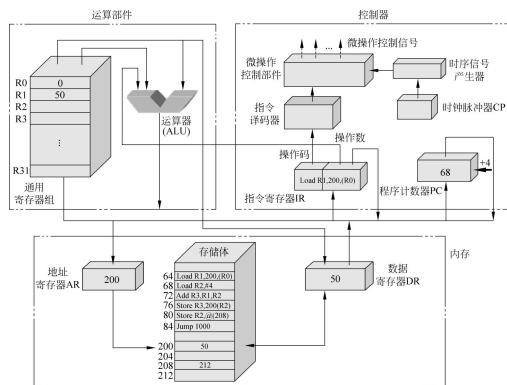
ALU 进行加法运算。结果传送给 AR，即：[ALU]→AR。如图 7 中的③所示。



从存储器读出数据，送入寄存器 R1。
具体操作：
从存储器中地址为 200 的存储单元读出数据 50，放到 DR。如图 8 中的①所示。
DR→R1。把 DR 中的 50 送入 R1。如图 9 中的②所示。

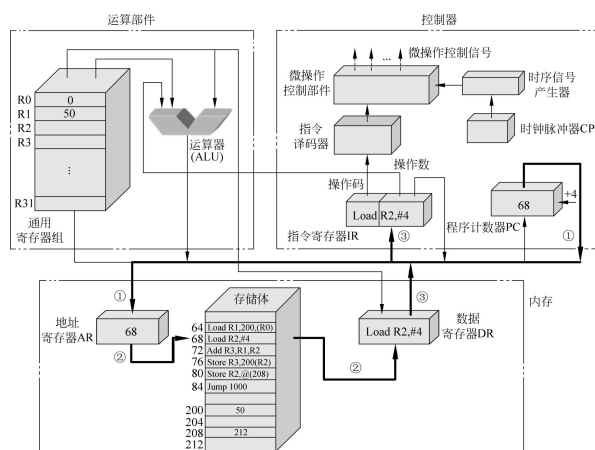


第 4 步：把 PC 中的地址加 4，指向下一条指令。为执行下一条指令做好准备。如图 10 中所示。



2. 第二条指令的执行过程

指令： Load R2, #4
功能：把指令中的立即数 4 传送给 R2
第 1 步：取指令
类似于上一条指令，只是 PC 的值为 68。



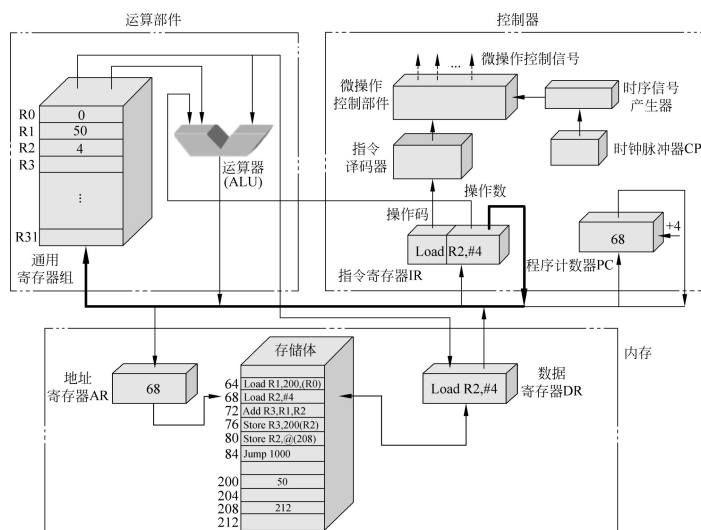
第 2 步：指令译码

控制器产生微操作控制信号，类似于第一条指令。

第 3 步：执行指令

将 IR 中的 4 传送到 R2。

第 4 步：同上一条指令。操作后 PC 的值为 72



3. 第三条指令的执行过程

指令：Add R3, R1, R2

功能：[R1]+[R2]→R3

把 R1 中的内容和 R2 中的内容相加，结果存入 R3。

第 1 步：取指令

操作步骤如图 13 中的所示。

第 2 步：指令译码

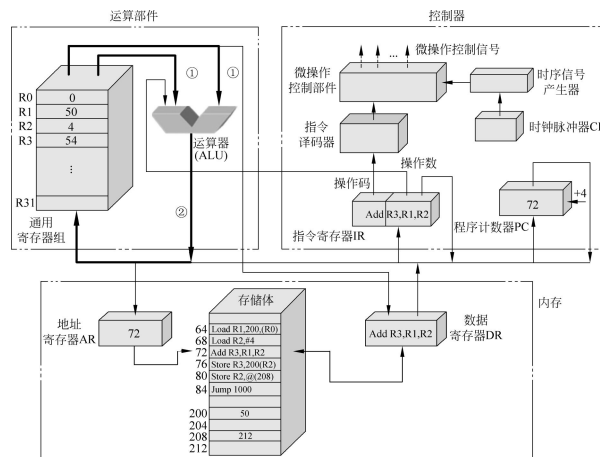
类似于第一条指令。

第 3 步：执行指令

把 R1 和 R2 中的数据送往 ALU 的两个入口端，ALU 做加法运算。如图 14 中的①所示。

把 ALU 出口端的结果存入 R3。如图 15 中的②所示。

第 4 步：同第 1 条指令。操作后 PC 的值为 76。



4. 第四条指令的执行过程

指令：Store R3, 200(R2)

功能：[R3]→MEM[[R2]+200]

把 R3 中的内容存入内存中地址为[R2]+200 的存储单元。

取指译码同前

第 3 步：执行指令

计算访存地址：[R2]+200→AR。

如图 17 中所示。

具体操作：

[R2]→ALU // 把寄存器 R0 中的内容（为 4）送给 ALU。

IR 中的操作数“200”→ALU。

ALU 的加法运算，[ALU]→AR // ALU 的运算结果（204）送给 AR。

[R3]→DR。

即把 R3 的内容送 DR。

如图 18 中所示。

.....到这里懒得整理了，请把第六章 PPT 通篇背诵吧

第 7 章 指令系统

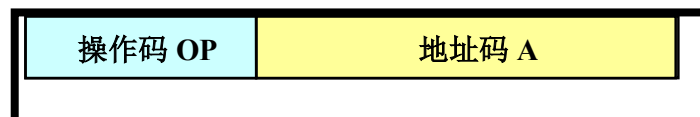
1. 什么是指令和指令系统，阐述指令的基本格式

指令：要求计算机进行基本操作的命令

指令系统：一台计算机所能执行的全部指令的集合称为该计算机的指令系统或指令集

指令的基本格式：

每条指令由两部分构成：操作码和地址码



操作码：描述指令的操作功能，即要求计算机完成什么工作。

地址码：描述与操作数有关的信息，可以是操作数本身，也可以是存放操作数的地址。

2. 指令的地址码

零地址、一地址、二地址、三地址和多地址

3. 指令的操作码

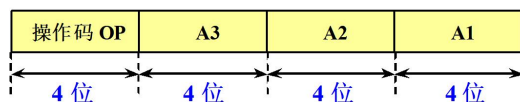
指令系统中的每一条指令都有唯一确定的操作码，不同指令的操作码是不相同的。

操作码的长度决定了指令系统的最大规模，若操作码的位数为 n 位，则该指令系统最多能有 2^n 条指令。

操作码的类型：

固定长度操作码、可变长度操作码、扩展操作码技术

4. 必考的指令操作码扩展



□ 采用固定长度操作码，该指令系统最多只能有16条指令。

➤ 在指令字长不变的情况下，希望有61条指令。

其中：

15条三地址指令

15条二地址指令

15条一地址指令

16条零地址指令

要会画出指令划分

下面这种情况又如何划分？

采用另一种扩展方法，共76条。

其中：

15条三地址指令

14条二地址指令

31条一地址指令

16条零地址指令

5. 表示操作数类型的方法

(1) 由指令中的操作码指定操作数的类型。

(2) 带标志符的数据表示。给数据加上标识，由数据本身给出操作数类型

优点：简化指令集，可由硬件自动实现一致性检查和类型转换，缩小了机器语言与高级语言的语义差距，简化编译器等。

缺点：由于需要在执行过程中动态检测标志符，动态开销比较大，所以采用这种方案的机器很少见。

6.（重点）寻址方式

直接寻址（绝对寻址）：在指令的地址码字段直接给出操作数所在主存单元的地址。

优点：简单、快速的寻址方式

缺点：寻址范围受限于地址码字段的位数

间接寻址：指令的地址码字段给出的是操作数所在内存单元的地址的地址。

缺点：得到操作数需要访问两次内存，指令的执行速度比较慢。

立即寻址：指令的地址码字段直接给出操作数本身，而不是操作数的地址。

优点：指令执行速度最快，得到指令的同时就得到了操作数，不需要再访问内存。

寄存器直接寻址：指令的地址码字段给出一个寄存器编号，该寄存器中存放的内容就是操作数。

寄存器间接寻址：寄存器中存放的内容是操作数的地址，根据此地址访问内存取得操作数。

相对寻址：将程序计数器 PC 的内容与指令中给出的形式地址（偏移量）的值相加，形成操作数的有效地址。（形式地址（偏移量）的值可正可负）

变址寻址：将指令中给出的形式地址的值与变址寄存器的内容相加，形成操作数的有效地址。（通常地址的变化体现在变址寄存器中，指令中的形式地址相对固定）

基址寻址：把指令中给出的形式地址的值与基址寄存器的内容相加，形成操作数的有效地址。（通常地址变化体现形式地址上，基地址相对不变。// 对比变址寻址）

必考类型习题：

某计算机字长为16位，运算器为16位，有16个16位通用寄存器，8种寻址方式，主存容量为64K字。指令中地址码由寻址方式字段和寄存器字段组成，采用单字长指令，则该计算机最多可构成(82)条单操作数指令；寄存器间接寻址的范围为(83)K字。

(2)

- A. 16
B. 32
C. 64
D. 128

正确答案

C

答案解析

[解析] 单操作数的指令长度为16位，其中，寻址方式为3位，16个通用寄存器占4位，其余9位用来构成指令操作码，共有 $2^9=512$ 种。所以，最多可构成512条指令。
第1空的正确答案为选项B。
因为通用寄存器为16位的，所以，寄存器间接寻址的范围为 $2^{16}/1024=64K$ 。第2空的正确答案为选项C。

- 2.13 某计算机字长16位，主存按字编址，采用单字长单地址指令格式，指令各字段定义如下：

15	12	11	9	8	6	5	0
OP-Code			M	Rn		A	

其中，OP-Code为操作码，M为寻址方式码，Rn为通用寄存器编号，A为形式地址。寻址方式码定义如下：

M	寻址方式	有效地址表达式
000B	一次间接	$EA = (A)$
001B	寄存器间接	$EA = (Rn)$
010B	变址	$EA = (Rn) + A$ ， $Rn \leftarrow (Rn) + 1$
011B	相对	$EA = (PC) + A$

注：有效地址表达式中(X)表示存储器地址X或寄存器X的内容；指令中Rn字段和A字段是否使用视寻址方式而定；位移量用补码表示。

请回答下列问题：

- (1)该指令系统最多可有多少条指令？该计算机最多有多少个通用寄存器？
- (2)上表中各种寻址方式的寻址范围多大（不包括相对寻址）？相对寻址的浮动范围多大？
- (3)设开始取指令时，对应寄存器和主存相关单元的内容如下图，图中的数字均为十六进制表示，请写出指令0627H和3559H的操作数各为多少？分别单独执行这两条指令后相关寄存器的内容各是多少？

	地址	主存
PC 2000H	19H	0100H
R0 0627H	27H	4000H
	400H	1000H
R5 0400H	401H	3559H
	419H	0123H
R7 3559H	41AH	0627H
	1FE7H	1234H
	1FE8H	5678H

□ 题解:

○1、该指令系统最多可有 $2^4=16$ 条指令，该计算机最多有

$2^3=8$ 个通用寄存器

○2、一次间接寻址范围= $2^{16}=64K$ 字、

寄存器间接寻址范围= $2^{16}=64K$ 字

变址寻址范围= $2^{16}=64K$ 字

相对寻址的浮动范围= $-32\sim+31$

○3、a、指令0627H展开: 0000 011 0 00 10 0111B

OP=0000B, M=011B=相对寻址,

Rn=000B (无用), A=10 0111B (负数补码)

EA=(PC)+A=2001H+FFE7H=1FE8H

(取指后(PC)+1, 且A符号扩展)

操作数a=(EA)= (1FE8H)=5678H

指令执行后: (PC)=2001H

○b、指令3559H展开: 0011 010 1 01 01 1001B

OP=0011B, M=010B=变址寻址,

Rn=101B (R5), A=01 1001B (正数补码)

EA=(R5)+A=0400H+0019H=0419H (A符号扩展)

操作数b=(EA)= (0419H)=0123H

(R5)=(R5)+1=0401H

指令执行后: (PC)=2001H, (R5)=0401H

7. 指令类型

数据传送类指令、数据运算类指令、程序控制类指令、输入输出指令

8. 指令系统的基本要求

完整性、规整性、正交性、高效率、兼容性

9. 三种指令编码格式

可变长度编码格式、固定长度编码格式、混合型编码格式

10. 几类指令的格式，结合联系第8章的数据通路

I类指令 (load、store、beqz)

包括所有的 load 和 store 指令，立即数指令，分支指令，寄存器跳转指令，寄存器链接跳转指令。

立即数字段为 16 位，用于提供立即数或偏移量

6		5		5		16						
操作码		rs		rt		立即数（immediate）						
0	5	6	10	11	15	16						31

Load: 访存有效地址: $\text{Regs}[\text{rs}] + \text{immediate}$, 从存储器取来的数据放入寄存器 rt (存储器->寄存器)

Store: 访存有效地址: $\text{Regs}[\text{rs}] + \text{immediate}$, 将寄存器 rt 中的数据存入存储器中 (寄存器->存储器)

立即数指令: $\text{Regs}[\text{rt}] \leftarrow \text{Regs}[\text{rs}] \text{ op immediate}$

分支指令: 转移目标地址: $\text{Regs}[\text{rs}] + \text{immediate}$, rt 无用

寄存器跳转、寄存器跳转并链接: 转移目标地址为 $\text{Regs}[\text{rs}]$

R 类指令

包括 ALU 指令, 专用寄存器读/写指令, move 指令等。

ALU 指令: $\text{Regs}[\text{rd}] \leftarrow \text{Regs}[\text{rs}] \text{ funct } \text{Regs}[\text{rt}]$, funct 为具体的运算操作编码

6		5		5		5		5		6	
操作码		rs		rt		rd		sham		funct	
0	5	6	10	11	15	16	20	21	25	26	31

J 类指令

包括跳转指令, 跳转并链接指令, 自陷指令, 异常返回指令。

在这类指令中, 指令字的低 26 位是偏移量, 它与 PC 值相加形成跳转的地址。

6		26									
操作码		与 PC 相加的偏移量									
0	5	6									31

11. 必考经典习题

第 7 章课后习题 10-17

第 8 章 中央处理器

1. 简述 CPU 的功能:

CPU 具有以下 4 个方面的基本功能: 指令顺序控制、操作控制、时间控制和顺序加工

2. 设计 CPU 中的控制器的技术有哪两种, 简述其思想

硬连逻辑: 硬连逻辑是建立在有限状态机的基础上, 并且一般是以状态图的形式表示。

微程序设计：微程序设计则是采用微指令的方式来表示和实现控制。

3. 控制器由哪几部分构成，每一部分的主要功能是什么？

指令部件：取指令和分析指令

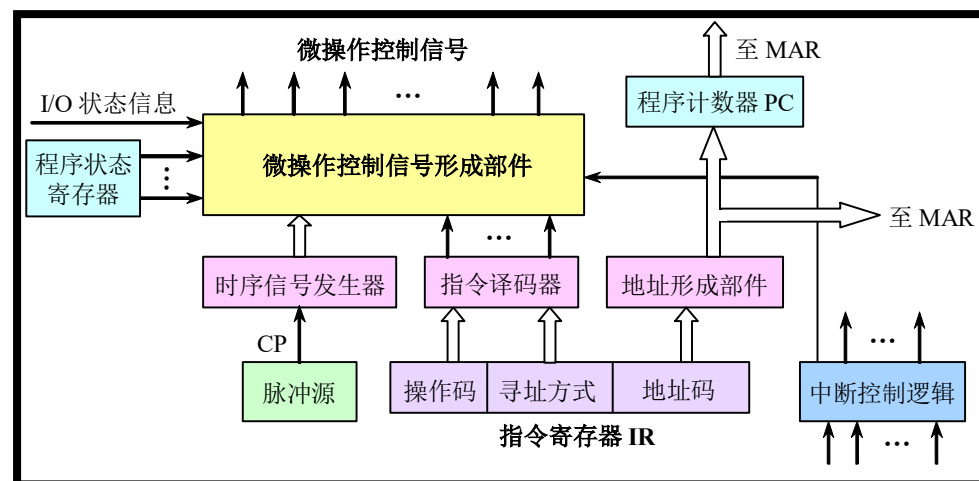
时序控制部件：用来产生一系列时序信号、为各个微操作定时的，以保证各个微操作的顺序执行

微操作控制信号形成部件：根据指令部件提供的操作控制电位、时序控制部件所提供的各种时序信号，以及有关的状态条件，产生计算机所需要的各种微操作的控制信号。

中断控制逻辑：又称中断机构，用于异常或突发情况的处理

程序状态寄存器：存放程序状态字 PSW

4. 控制器的组成图



5. 控制器的控制方式通常分为哪两种实现方法，各有什么优缺点

同步控制方式

缺点：有时间浪费。

优点：时序关系简单，控制方便，而且便于调试，系统较为可靠。

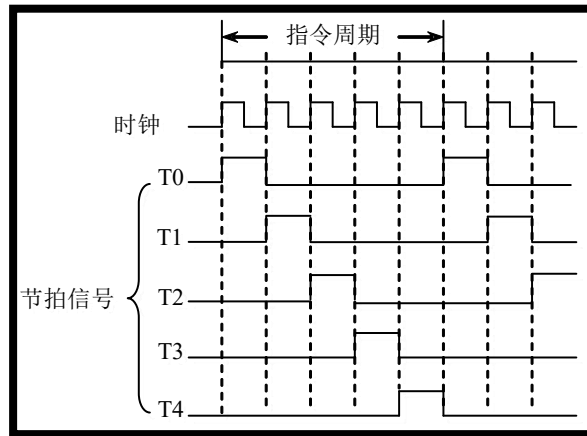
异步控制方式

优点：没有时间上的浪费，效率高。

缺点：设计比较复杂，所需的器材较多，系统调试难度较大，且工作过程中的可靠性不易保证。

6. 用图示说明指令周期、节拍、脉冲之间的关系

其中每个指令周期包含 5 个节拍 T0~T4，每个节拍有一个脉冲

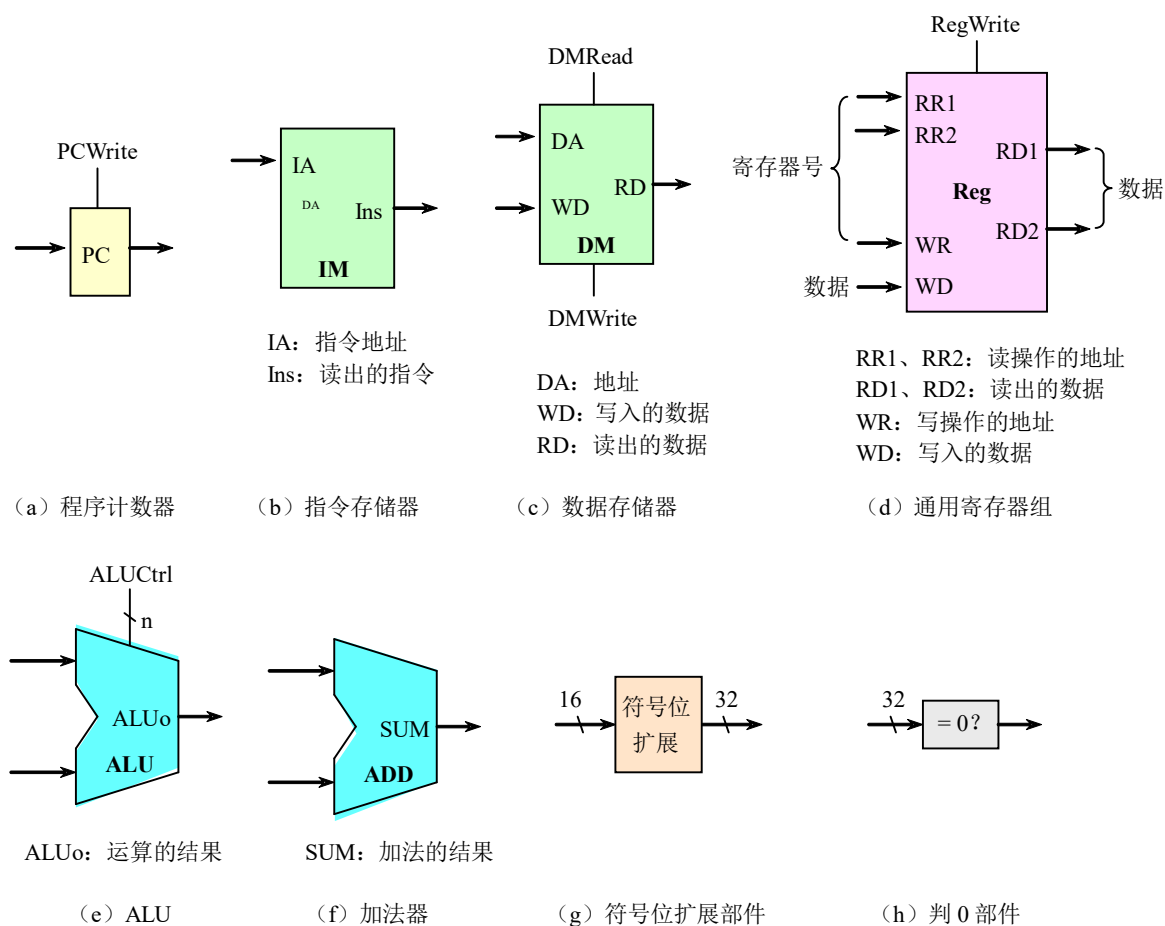


3 级时序系统

7. 为什么现在的中央处理机多采用多周期的设计方案

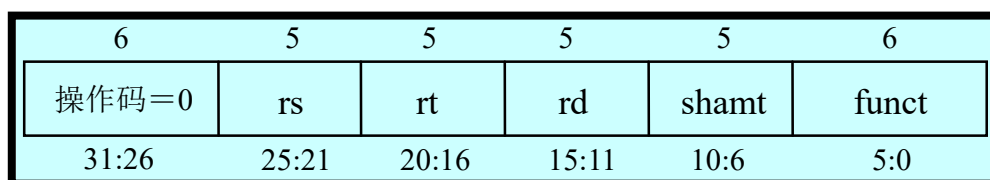
单周期方案效率低下，如果采用单周期，那么周期时间只能取最长数据通路花费时间，会造成时间浪费；每个时钟周期中功能部件最多被使用一次，如果要在执行一条指令的过程中，多次使用某一部件，那么就需要重复设置该部件，会增加实现成本；而采用多周期方案，可以采用更短的时间作为时钟周期，允许指令的执行时间为多个时钟周期；可以共享同一个功能部件（如果是在不同的时钟周期使用该部件的话）

8. 构造数据通路的 8 种基本构件



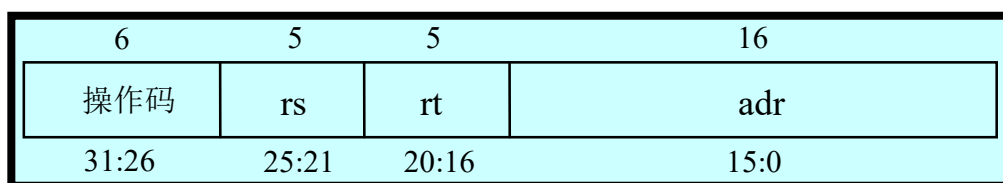
9. 划分出下列指令的格式

(1) R 类指令



Op——操作码字段，用 IR[Op]或 IR[31:26]表示；
rs——第一源操作数字段，用 IR[rs]或 IR[25:21]表示；
rt——第二源操作数字段，用 IR[rt]或 IR[20:16]表示；
rd——目标操作数字段（或结果字段），用 IR[rd]或 IR[15:11]表示；
shamt——无用；
funct——ALU 指令的运算函数码字段，用 IR[funct]或 IR[5:0]表示。

(2) I 类指令



rs——基址寄存器字段，用 IR[rs]或 IR[25:21]表示。对于 beqz 指令来说，是存放被检测的数据。

adr——偏移量字段，用 IR[adr]或 IR[15:0]表示。rs 和 adr 用于计算访存有效地址或分支目标地址。

rt——对于 load 指令来说，rt 所指出是用来存放所取的数据的寄存器；对于 store 指令来说，rt 所指出的是存放要写入存储器的数据的寄存器。

(3) beqz 指令

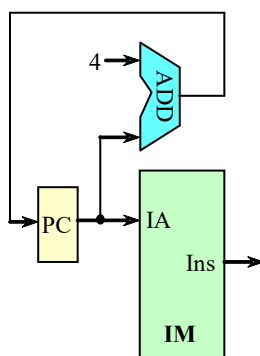
等于“0”分支（I 类型指令格式）

beqz, Op=63

说明：beqz 在 MIPS 中实际上是条伪指令。（假设模型机中有该指令，并假设其操作码为 63）

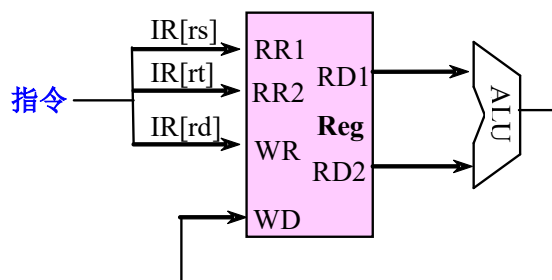
10. 画出下列简单数据通路

(1) 取值数据通路



描述：把 PC 中的地址送到指令存储器 IM 的 IA 输入端，读出一条指令。同时用加法器把 PC 中的值加 4，使它指向下一条指令。(注：在第四个节拍 T3 时写入 PC，不一定是 PC+4，也可能是在 beqz 指令中得到的跳转地址)

(2) R 类指令数据通路



描述：

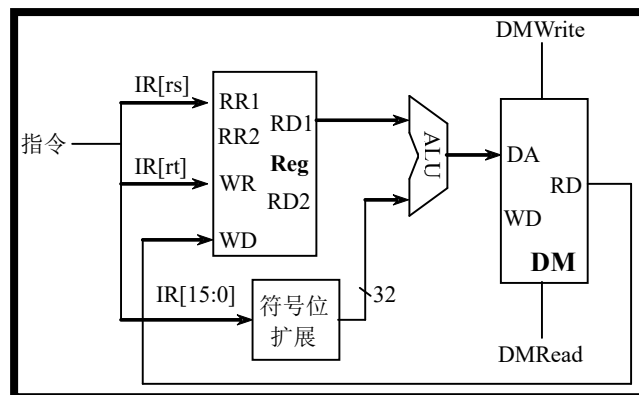
用指令中的源寄存器地址字段 rs 和 rt 作为地址去访问通用寄存器组 Reg，读出两个源

操作数，送给 ALU 进行运算。

把运算结果送到寄存器组的 WD 端，写入由 rd 字段所指定的目标寄存器。

(3) 访存指令(load 和 store)

Load 指令（从有效地址取出数据存放到 rt 指定寄存器）



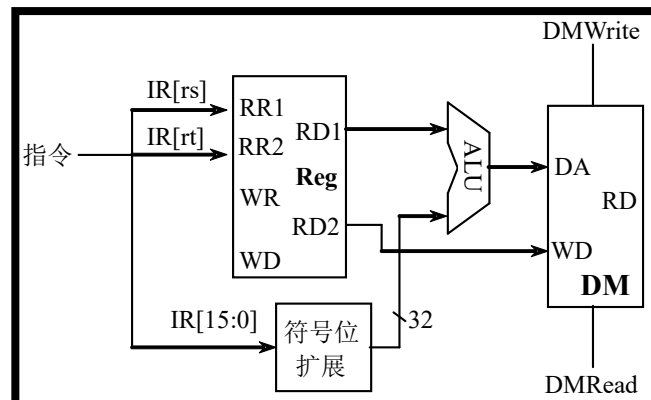
描述：

把指令字中的 16 位偏移量字段进行符号位扩展，变成 32 位数，送给 ALU；

用 IR[rs]作为地址去访问寄存器组 Reg，读出的操作数送给 ALU，与上一步扩展了的地址相加，计算出访存的有效地址，将之送到数据存储器 DM 的地址输入端 DA；

从 DM 读出数据（将 DMRead 设置为有效），将该数据送到通用寄存器组的数据输入端口 WD，写入由 IR[rt]指定的寄存器。

Store 指令（向有效地址直接存放 rt 指定寄存器中的数据）

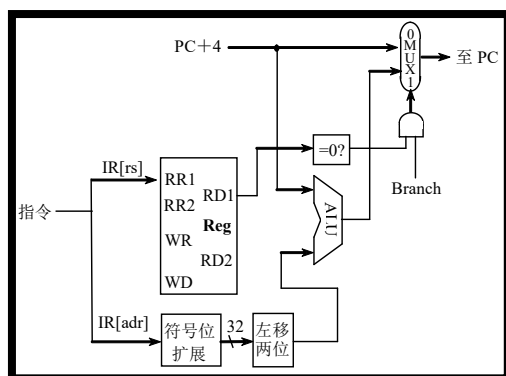


描述：

前两步与 load 指令的相同。

用 IR[rt]作为地址去访问通用寄存器组，读出的数据（在 RD2 输出端口）送给 DM 的数据输入端 WD，并向 DM 发写入信号（将 DMWrite 置为有效），将数据写入 DM 中相应单元（也就是 rs 和拓展地址相加得到的有效地址对应的单元）。

(4) beqz 指令



Branch 信号为有效表示当前指令是一条分支指令。

把指令字中的 16 位偏移量字段（IR[adr]）进行符号位扩展，变成 32 位数，并左移两位送给 ALU；

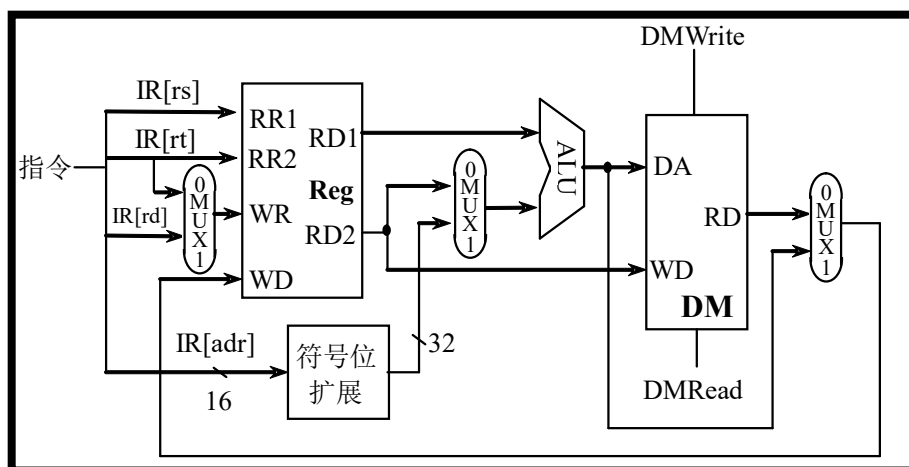
把 PC+4 送给 ALU 的另一个输入，与上一步符号位扩展和左移后的地址相加，得到转移目标地址；

用 IR[rs]作为地址去访问寄存器组 Reg，读出操作数并送给判 0 部件。由该部件的输出确定是否分支成功。

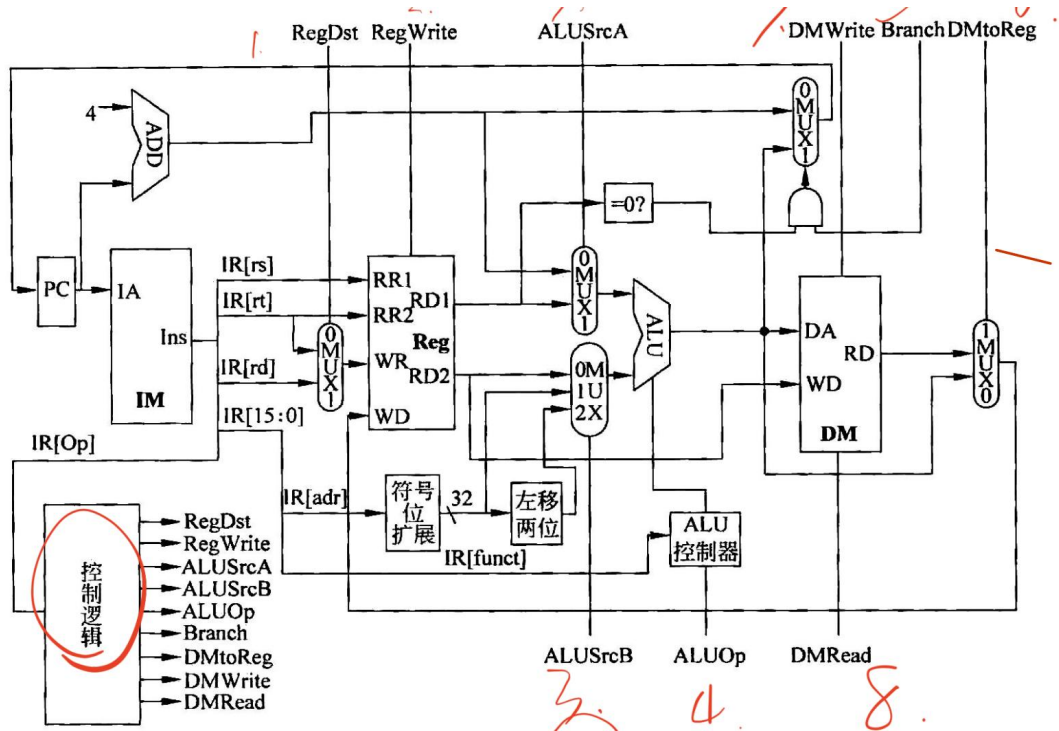
如果为真，转移目标地址就成为新的 PC 值，分支成功；否则就用 PC+4 代替 PC 中的值，分支失败。

11. 组合类数据通路

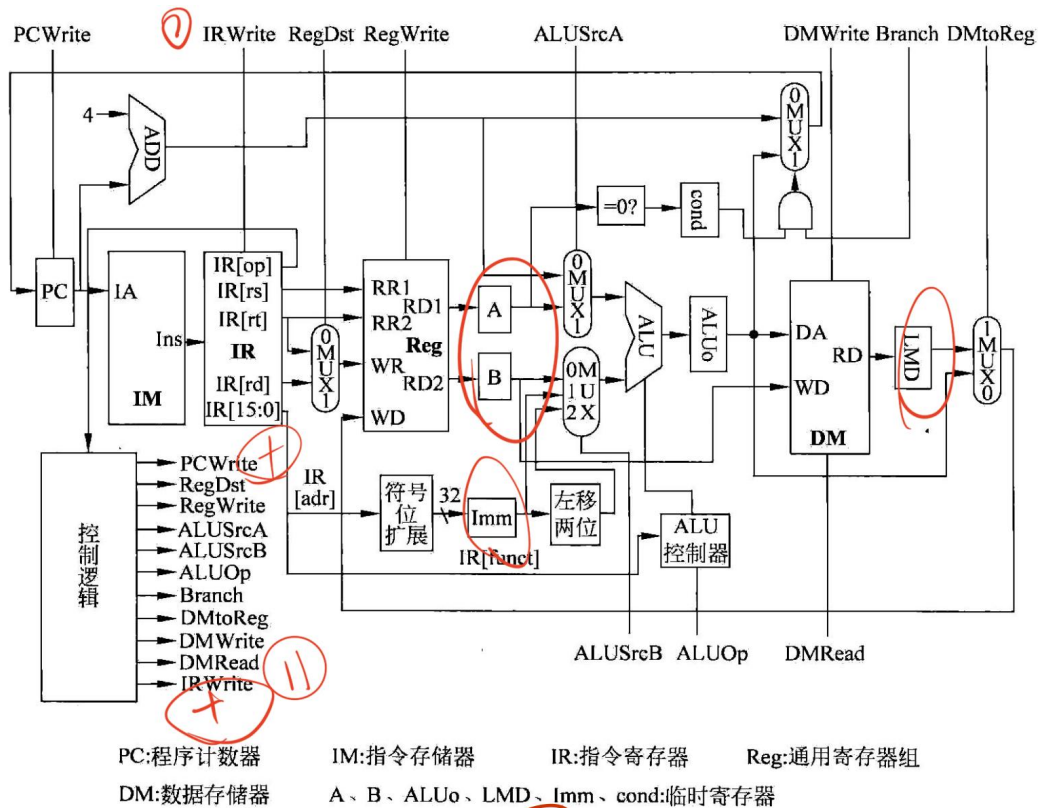
访存指令和 R 类指令数据通路：



单周期模型机的数据通路：



多周期模型机的数据通路



PC:程序计数器

IM:指令存储器

IR:指令寄存器

Reg:通用寄存器组

DM:数据存储器

A、B、ALUo、LMD、Imm、cond:临时寄存器

图 8.14 模型机多周期实现的数据通路

12. 指令分步执行过程

周期名称	R类指令 Op= “R类”	存储器访问指令 Op= “lw” 或Op= “sw”	分支指令 Op= “beqz”
T0:取指令 (IF)	操作: $IR \leftarrow IM[PC]$ $PC \leftarrow PC+4$ 控制信号: $IRWrite=1$ //把新取出的指令写入指令寄存器IR		
T1:指令译码/ 读寄存器 (ID)	操作: $A \leftarrow Regs[rs]$ $B \leftarrow Regs[rt]$ $Imm \leftarrow (IR[15:0])$ 按符号位扩展为32位数) 控制信号: 不需要 注: 指令的译码操作和读寄存器操作是并行进行的。 原因: 在MIPS指令格式中, 操作码字段以及rs、rt 字段都是在固定的位置。这种技术称为固定字段译码技术。		
T2:执行/有效地址 计算 (EX)	操作: $ALUo \leftarrow A \text{ funct } B$ 控制信号: $ALUSrcA=1$ //选择A $ALUSrcB=00$ //选择B $ALUOp=10$ //按照函数码funct规定的操作进行运算	操作: $ALUo \leftarrow A+Imm$ 控制信号: $ALUSrcA=1$ $ALUSrcB=01$ //选择Imm(拓展地址) $ALUOp=00$ //加法	操作: $ALUo \leftarrow PC + (Imm \ll 2)$ $cond \leftarrow (A == 0)$ 控制信号: $ALUSrcA=0$ //选择PC+4 $ALUSrcB=10$ //选择Imm左移两位 $ALUOp=00$
周期名称	R类指令 Op= “R类”	存储器访问指令 Op= “lw” 或Op= “sw”	分支指令 Op= “beqz”
T3:存储器访问/R类和分支 指令完成 (MEM)	操作: $Regs[rd] \leftarrow ALUo$ 控制信号: $DMtoReg=0$ //选择ALUo中的内容 送到寄存器组的WD端。 $RegDst=1$ //选择rd作为写入地址 $RegWrite=1$ //写入寄存器 $PCWrite=1$ //写入PC	load指令: 操作: $LMD \leftarrow DM[ALUo]$ 控制信号: $DMRead=1$ //从DM读取数据 $PCWrite=1$ store指令: 操作: $DM[ALUo] \leftarrow B$ 控制信号: $DMWrite=1$ //向DM写入数据 $PCWrite=1$	操作: If (cond&Branch) $PC \leftarrow ALUo$ else $PC \leftarrow PC+4$ 控制信号: $Branch=1$ //表示当前指令是分支指令 $PCWrite=1$
T4:写回 (WB)		操作: load指令: $Regs[rt] \leftarrow LMD$ 控制信号: $DMtoReg=1$ //选择LMD中的内容 送到寄存器组的WD端 $RegDst=0$ //选择rt作为写入地址 $RegWrite=1$	

13. 解释下列术语

流水线技术: 把一个重复的过程分解为若干个子过程, 每个子过程由专门的功能部件来实现。把多个处理过程在时间上错开, 依次通过各功能段, 这样, 每个子过程就可以与其它的子过程并行进行。

通过时间: 第一个任务从进入流水线到流出结果所需的时间。

排空时间: 最后一个任务从进入流水线到流出结果所需的时间。

吞吐率：在单位时间内流水线所完成的任务数量或输出结果的数量。

$$TP = \frac{n}{T_k}$$

n: 任务数

Tk: 处理完成 n 个任务所用的时间

流水线加速比：完成同样一批任务,不使用流水线所用的时间与使用流水线所用的时间之比。

$$S = \frac{T_s}{T_k}$$

不使用流水线（即顺序执行）所用的时间为 Ts，使用流水线后所用的时间为 Tk

流水线的效率：流水线中的设备实际使用时间与整个运行时间的比值，即流水线设备的利用率。

(由于流水线有通过时间和排空时间，所以在连续完成 n 个任务的时间内，各段并不是满负荷地工作。)

第 9 章 微程序控制器

1. 基本概念

微命令(微操作控制信号): 构成控制信号序列的最小单元, 由控制部件通过控制线向执行部件发出

微操作: 指执行部件接受控制信号后所进行的最基本的、不可再细分的操作, 分为相容和相斥的微操作。

微指令: 用来产生微控制信号的二进制编码字。用于控制完成一组微操作。

微程序: 一序列微指令构成的有序集合。每一条机器指令都对应于一段微程序, 通过解释执行这段微程序, 完成指令所规定的操作。

微指令周期: 微程序控制器的工作周期。从控制存储器读取一条微指令到执行完相应的微操作所需时间的最大值。

2. 微程序控制的基本原理（基本思想）

用二进制编码字（称为微指令字）来代替组合逻辑控制器中的微操作控制信号的产生。实质是用程序设计的思想方法来组织操作控制逻辑, 用规整的存储逻辑来代替繁杂的组合逻辑。

3. 微程序控制器的组成部分

控制存储器 CM, 简称控存, 用于存放实现整个指令系统的所有微程序。每个单元存放一条微指令字。

微指令寄存器 μIR , 用来存放从控存读出的当前微指令。

微指令中包含两个字段:

操作控制字段: 直接与控制信号线连接, 控制相关的部件完成微指令所规定的微操作。

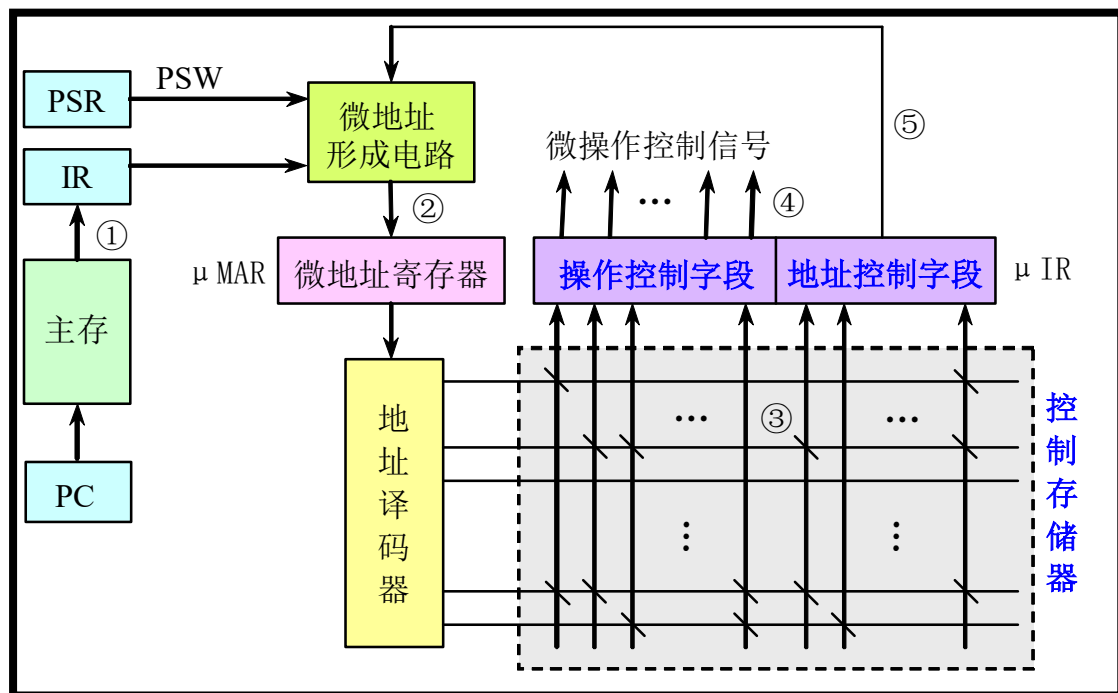
地址控制字段: 用于控制下一条微指令地址的产生

微地址形成电路, 该电路根据控制地址字段中的信息产生后续微地址。

微地址寄存器 μMAR , 接受微地址形成电路送来的地址, 为读取下一条微指令做好准备。

地址译码器, 将 μMAR 中的微地址进行译码, 找到被访问的控存单元, 将其中的微指令读出并存放于 μIR 中。

4. Wilkes 微程序控制原理图



5. 简述微程序控制器的工作过程

(1) 启动取指微程序，把要执行的机器指令（其地址由 PC 给出）从主存取到指令寄存器 IR 中，并完成对 PC 的增量操作（参照图中的①）。

(2) 根据 IR 中指令的操作码，微地址形成电路产生该指令的微程序的入口地址，并送入 μMAR。

(3) μMAR 中的微地址经过译码，从控存中读取相应的微指令，送入 μIR 。

(4) μIR 中微指令的操作控制字段直接（或经过译码）产生一组微命令，送往相应的功能部件，控制它们完成所规定的微操作。

(5) 微地址形成电路根据 μIR 中微指令的地址控制字段和机器的状态信息（如程序状态字 PSW），产生下一条微指令的地址并送往 μMAR。

(6) 重复上述步骤第三到第五步，直到该机器指令的微程序全部执行完毕。

6. 对微操作字段进行编码的四种方法

(1) 直接控制编码（不译码法）：

微操作控制字段的每一位直接对应一个微操作

优点：结构简单，并行性最好，操作速度快。

缺点：微指令字太长。

(2) 最短字长编码法：

将所有的微命令进行统一的二进制编码，每条指令只定义一个微操作。

微操作控制字段的长度 L 与微命令的总数 N 的关系： $L \geq \log_2 N$

优缺点：微指令的字长最短，但要经过译码后才能得到所需要的微命令，执行速度会受到影响。在一条微指令中只能产生一个微命令，无法利用硬件所具有的并行性。

(3) 字段直接编码法:

把微操作控制字段进一步划分为若干个字段,每个字段单独编码,每个码点表示一个微命令
字段之间采用直接控制,字段内部采用最短字长编码

编码原则:

可以按功能和部件划分,对于机器中的每一种功能类型或每一个部件,分配一个字段;

把互斥的微操作分在同一字段,把相容的微操作分到不同的字段 ;

字段的划分应与数据通路相适应;

一般每个字段应留出一个码点,用于表示不发任何微命令。

优点:既能缩短微指令字长,又能实现较高的并行性,执行速度比较快。

(4) 字段间接编码法

字段的编码的含义(即表示什么微命令)要由另外一个字段的编码来解释确定。

一个解释字段要同时对多个字段进行控制(解释),才能有效地缩短字长。

解释字段应有某些分类的特征

(5) 常数源字段的设置

提供常数、和机器指令中的立即数作用一样,

参与其他字段的间接编码。

7. 微指令格式

分为两大类:水平型微指令和垂直型微指令。

水平型微指令,一次能定义并执行多个微操作的微指令。

具有以下特点:

微指令字较长,一般为几十位到上百位。

微指令中描述并行微操作的能力强,在一个微周期中,能并行执行多个微操作。

微指令译码简单,一般采用直接控制编码法和分段直接编码法。

优缺点:

并行操作能力强、执行速度快、代码长度短。

微指令字比较长,明显增加了控存的宽度。

编制微程序比较复杂,难度较大,也不易实现设计的自动化。

垂直型微指令,一次只能定义一两个微操作(一般是数据传送),而且微指令字长比较短。

特点:

微指令字短,一般为一、二十来位;

微指令的并行微操作能力差,一条微指令只能控制数据通路的一两种信息传送;

垂直型微指令是通过一个称为微操作码的字段来定义微指令的基本功能和信息传送路径。执行时,需进行完全译码,译码比较复杂。

微指令的各二进制位与数据通路的各个控制点之间完全不存在直接对应关系。

优点:

直观、规整,易于编制微程序和实现设计自动化。

微指令字比较短,控存的横向比较窄。

可以直接应用现有程序设计语言的结果。

缺点:

微程序比较长,而且垂直型微指令产生微命令要经过译码,程序执行速度慢。

描述并行微操作的能力很差,不适合用于数据通路有较多并行性的机器。

8. 微程序的顺序控制中，产生后继微地址的方式有哪些

两种方式：增量方式，断定方式。

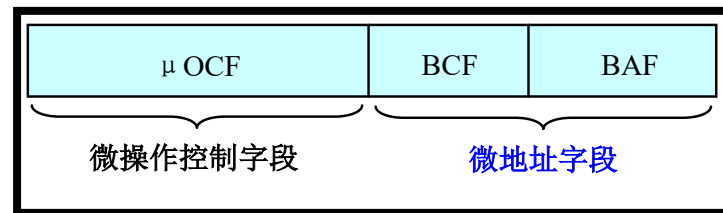
增量方式

设置一个微程序计数器 μPC 。

顺序执行时，给 μPC 增加一个增量（通常为 1）给出下一条微指令的地址。

遇到转移时，由微指令给出转移目标的微地址。

采用这种方式的微指令格式：



微地址字段 SCF 分成两个字段

转移控制字段 BCF 用于规定是顺序执行还是转移。如果是转移，就由 BCF 指出转移地址的来源。

转移地址字段 BAF

转移地址的来源有 3 种：

由 BAF 给出的地址；

机器指令所对应的微程序的入口地址；

微子程序入口地址和返回地址（存放在返回地址寄存器中）。

优点：SCF 字段比较短，后继微地址生成逻辑比较简单，编制微程序也比较容易。

缺点：不能直接实现多路转移。

断定方式

按以下方式确定后继微地址：

由微程序设计者直接指定；

由微程序设计者指定的测试判别逻辑字段控制产生。

后继微地址由两部分组成：

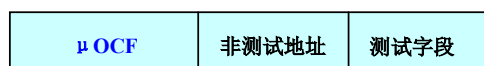
非测试地址，微程序设计者直接指定的，是不变的，构成微地址的高位部分。

测试地址，在微程序的执行过程中，通过测试一些状态位而动态决定的，它构成微地址的低位部分。

微地址格式



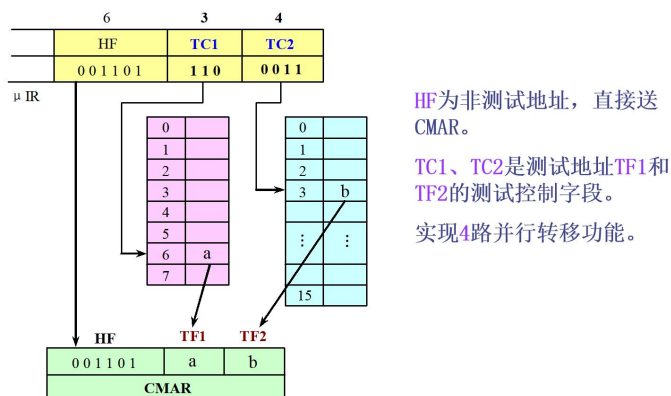
微指令格式



当测试地址位数为 m 时，分支的路数为 2^m ，而测试字段的个数为 m 。至于测试字段的位数 n ，则是取决于测试条件的个数 N ，一般地，有 $n = \lceil \log_2 N \rceil + 1$ 。

例如：

例如：微地址8位，微程序能实现4路并行转移。可控制微程序的测试条件共23个，分成2组，第一组8个，第二组15个。



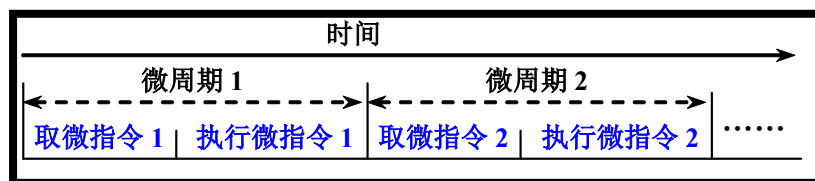
优点：能够实现快速多路转移，提高微程序的执行速度。而且微程序在控存中的存放位置也很灵活、方便。

缺点：后继微地址的生成方法比较复杂，微程序的执行顺序不直观。

9. 微指令的执行方式

串行执行方式：取微指令和执行微指令串行进行。在前一条的微指令执行完之后，才能取下一条微指令

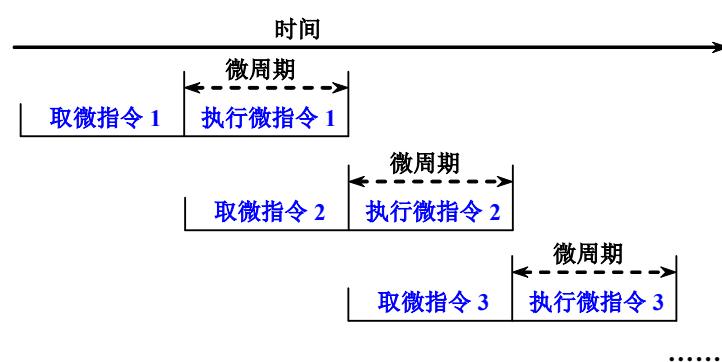
时间关系：



优缺点：设备效率低，执行速度慢。控制简单，易于实现。

并行执行方式：当前微指令的执行和下一条微指令的取出重叠进行。

时间关系：



优点：提高了执行速度和设备利用率

10. 必考大题

9.10 某计算机有 8 条微指令 $I_1 \sim I_8$ ，每条微指令所包含的微操作见如题表 9.10 所示， $a \sim j$ 分别对应 10 种不同性质的微操作信号。假设微指令的微操作控制字段不超过 8 位，请安排微指令的控制字段格式。

题表 9.10

微指令	a	b	c	d	e	f	g	h	i	j
I_1	✓	✓	✓	✓	✓					
I_2	✓			✓		✓	✓			
I_3		✓						✓		
I_4			✓							
I_5			✓		✓		✓		✓	
I_6	✓							✓		✓
I_7			✓	✓				✓		
I_8	✓	✓						✓		

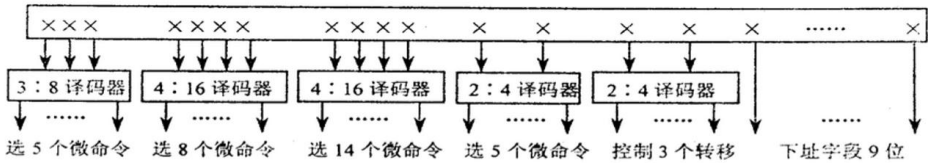
互斥

3. 某机采用微程序控制方式，微指令字长 24 位，水平型编码控制的微指令格式，断定方式，共有微命令 30 个，构成 4 个相斥类，各包含 5 个、8 个、14 个和 3 个微命令，外部条件共 3 个。

- (1) 控制存储器的容量应为多少？
- (2) 设计出微指令的具体格式。

解：

(1) 30 个微命令构成 4 个相斥类，其中 5 个相斥微命令需 3 位编码；8 个相斥微命令需 4 位编码，14 个相斥微命令需 4 位编码，3 个相斥微命令需 2 位编码；外部条件 3 个，采用断定方式需 2 位控制位。以上共需 15 位。微指令字长 24 位，采用水平型编码控制的微指令格式，所以还剩 9 位作为下址字段，这样控制存储器的容量应为 512×24 。



第 10 章 运算方法和运算器（必考计算题）

1. 移位运算

逻辑移位：逻辑左移，高位丢弃，低位补 0；逻辑右移，低位丢弃，高位补 0

循环移位

算术移位：

原码：符号位不动，其余位和逻辑移位一致

补码：符号位跟着移动。左移，高位丢弃，低位补 0；右移，高位补符号位，低位丢弃

2. 补码的加减法运算（重点）

补码加法时，将两个补码数直接相加，即得两数之和的补码；

补码减法运算转化为加法运算进行，将 $[X]_{\text{补}}$ 与 $[-Y]_{\text{补}}$ 相加，即得两数之差的补码；

至于 $[-Y]_{\text{补}}$ 的求法就是 $[Y]_{\text{补}}$ 取反加 1

如果运算结果超过了模（即符号位运算产生了进位），则将该进位自动丢弃。

例题：

例1 $[X]_{\text{补}} = 01001$ ， $[Y]_{\text{补}} = 00101$ ，求 $[X+Y]_{\text{补}}$ ， $[X-Y]_{\text{补}}$ 。

解：
 $[X+Y]_{\text{补}} = [X]_{\text{补}} + [Y]_{\text{补}} = 01001 + 00101 = 01110$
 $[X-Y]_{\text{补}} = [X]_{\text{补}} + [-Y]_{\text{补}} = 01001 + 11011 = 00100$

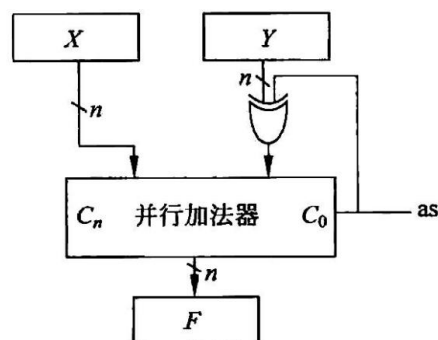
例2 $[X]_{\text{补}} = 01001$ ， $[Y]_{\text{补}} = 11011$ ，求 $[X+Y]_{\text{补}}$ ， $[X-Y]_{\text{补}}$ 。

解：
 $[X+Y]_{\text{补}} = [X]_{\text{补}} + [Y]_{\text{补}} = 01001 + 11011 = 00100$
 $[X-Y]_{\text{补}} = [X]_{\text{补}} + [-Y]_{\text{补}} = 01001 + 00101 = 01110$

例3 $[X]_{\text{补}} = 11010$ ， $[Y]_{\text{补}} = 11101$ ，求 $[X+Y]_{\text{补}}$ ， $[X-Y]_{\text{补}}$ 。

解：
 $[X+Y]_{\text{补}} = [X]_{\text{补}} + [Y]_{\text{补}} = 11010 + 11101 = 1011$
 $[X-Y]_{\text{补}} = [X]_{\text{补}} + [-Y]_{\text{补}} = 11010 + 00011 = 11101$

硬件实现：



X 、 Y :被加数、加数 F :和
 as :加减控制信号

图 10.7 补码加减法运算的电路

As 为 0 时做加法，为 1 时做减法

3. 溢出判断

如果两个正数相加的结果大于机器所能表示的最大正数，则称为正溢出。

如果两个负数相加的结果小于机器所能表示的最小负数，则称为负溢出。

判断溢出的 3 种方法（最常用变形补码）：

1. 采用两个操作数和结果的符号来判断
2. 采用最高数值位产生的进位与符号位产生的进位是否相同来判断。当不相同，则发生了溢出。

3. 采用变形补码来判断（采用两位符号位表示的补码叫变形补码）

若结果符号位为 01，则表示结果为正溢出；

若结果符号位为 10，则表示结果为负溢出。

$$\begin{array}{r} [X]_{\text{变形补}} \quad 00 \ 1010 \\ +[Y]_{\text{变形补}} \quad 00 \ 1001 \\ \hline 01 \ 0011 \end{array}$$

结果的符号为 01，溢出

$$\begin{array}{r} [X]_{\text{变形补}} \quad 11 \ 0000 \\ +[Y]_{\text{变形补}} \quad 11 \ 1111 \\ \hline 10 \ 1111 \end{array}$$

结果的符号为 10，溢出

4. 原码一位乘法（重点）

（1）符号位单独处理。用被乘数和乘数的数值位部分进行运算。设部分积的初值为 0。

（2）若乘数 Y 的最低位 $Y_0=0$ ，则将上一次的部分积右移一位，得新的部分积；

（3）若乘数 Y 的最低位 $Y_0=1$ ，则将上一次的部分积与被乘数 X 相加，然后右移一位（高位补 0），得新的部分积。

（4）乘数 Y 循环右移一位。

（5）将上述过程（2）重复 n 遍，n 为被乘数和乘数数值位的位数。最后得到的部分积就是乘积的数值部分

看例子

例7 已知 $[X_fX]_{\text{原}}=01101$ ， $[Y_fY]_{\text{原}}=11011$ ， $n=4$ ，

求 $[U_fUV]_{\text{原}}=[X_fX]_{\text{原}} \times [Y_fY]_{\text{原}}=?$

解： 符号位 $U_f=0 \oplus 1=1$

得： $[U_fUV]_{\text{原}}=[X_fX]_{\text{原}} \times [Y_fY]_{\text{原}}=110001111$

步数n	操作	进位C	部分积UV		乘数Y
初值		0	0000	××××	1011
4	$Y_0=1, +X$	0	1101	××××	
	C和UV一起右移一位 Y循环右移一位	0	0110	1×××	1101
3	$Y_0=1, +X$	1	0011		
	C和UV一起右移一位 Y循环右移一位	0	1001	11××	1110
2	$Y_0=0$ ，不做加法				
	C和UV一起右移一位 Y循环右移一位	0	0100	111×	0111
1	$Y_0=1, +X$	1	0001		
	C和UV一起右移一位 Y循环右移一位	0	1000	1111	1011

5. 定点运算器的构成（应该了解就行）

算术逻辑运算单元 ALU 由逻辑运算部件、算术运算部件以及多路选择器 3 部分组成。

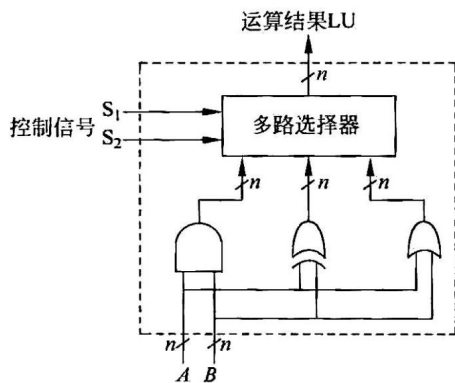


图 10.22 n 位的逻辑运算部件

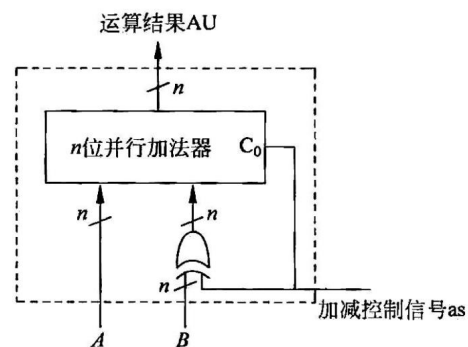


图 10.23 n 位的算术运算部件

6. 浮点运算（重点）

(1) 判 0：有 0 就不用算了

(2) 对阶：求阶差 $\Delta E = XE - YE$

若 $\Delta E > 0$ ，则表示 X 的阶码大于 Y 的阶码，需调整操作数 Y 。

将 Y 的尾数 Y_M 右移；

每右移一位，其阶码 Y_E 加 1，直到两数的阶码相等为止。

小于 0 就右移 X 的尾数

(3) 尾数加减

(4) 规格化，判断溢出：

什么时候右移：双符号位不同，正负溢出时都右移 1 位，双符号位一起右移，至于最低位采用什么舍入方式要看题目的要求

什么时候左移：符号位和数值第一位相同，左移到不同为止，低位补 0，高位直接舍弃

注意在左右移动的时候要同时加减阶码

在规格化时，阶码每次加 1 或减 1 以后，都要判断阶码是否超出所能表示的范围。

舍入方式

0 舍 1 入法，如果右移后的最低位是 0 就直接舍弃，如果是 1 就进 1

截断法，不管三七二十一，直接丢弃

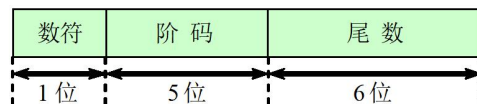
朝 $+\infty$ 舍入法：朝着变大的方向选择进 1 还是舍弃，但如果后面的位数都是 0 就直接截断

朝 $-\infty$ 舍入法：朝着变小的方向选择进 1 还是舍弃，但如果后面的位数都是 0 就直接截断

看例子：（要时刻注意什么时候要取反加 1）

例13 已知 $X = +0.110101 \times 2^{+0011}$, $Y = -0.111010 \times 2^{+0010}$,
求 $X \pm Y$ 。

假设浮点数格式为：



阶码、尾数均采用补码表示，阶码用双符号位，尾数用单符号位。
舍入处理采用简单的截断法。

解： 将X和Y转化成规格化浮点表示：

X: 0 00011 110101

Y: 1 00010 000110

这里取反加 1 了

(1) 求阶差并对阶

$$[\Delta E]_{\text{补}} = 00011 + 11110 = 00001 > 0$$

调整Y：将Y的尾数1000110右移一位（按补码算术右移规则），
阶码加1，得：

Y: 1 00011 100011

运算结果的阶码为：00011

(2) 尾数加/减

运算时，为了能保存结果的正确符号位，采用变形补码。

$$[X_M + Y_M]_{\text{补}} = 00110101 + 11100011 = 00011000$$

$$[X_M - Y_M]_{\text{补}} = [X_M]_{\text{补}} + [-Y_M]_{\text{补}} = 00110101 + 00011101 = 01010010$$

这里取反加 1 了，双符号位跟着一起

(3) 规格化并判溢出

$[X_M + Y_M]_{\text{补}}$ 的结果为00011000，需要将尾数左移一位，同时阶
码减1，没有溢出，得：

$X + Y$: 0 00010 110000 即 $+0.110000 \times 2^{+0010}$

符号位和数值第一位相同，这里只
需要左移 1 位

$[X_M - Y_M]_{\text{补}}$ 的结果为01010010，需要将尾数右移一位，同时阶
码加1，没有溢出，得：

$X - Y$: 0 00100 101001 即 $+0.101001 \times 2^{+0100}$

符号位不同，需要右移 1 位

注意从规格化浮点表示结果写
回去的时候，如果是负数，就还
得再取反加 1 回去，这个例子没
有是因为恰好都是正数

双符号位只在尾数加减的步骤和判断是否左右
移的时候使用，其他的时候又恢复成单符号位了

第 11 章 存储器

1. 课后文档要点（待复习补充）：

- （1）cpu 能直接访问的最大的存储器是哪个：主存
- （2）计算机的存储系统为什么采用层次化结构？：为了平衡速度、容量和成本之间的矛盾。通过将快速但昂贵、容量较小的存储器（如 RAM）与容量大但速度较慢、成本较低的存储器（如硬盘）结合使用，可以在保持成本合理的同时，提供既快速又大容量的存储解决方案
- （3）什么是破坏性读：读操作后需要进行恢复工作。 恢复：再给电容 C 充电，相当于进行一次写“1”的操作
- （4）

2. 存储器的分类

- （1）按在计算机系统中的作用分类
主存储器、辅助存储器、高速缓冲存储器 Cache
- （2）按存取方式分类（存取方式是指访问存储单元的方法）
随机存储器 RAM（又称易失性存储器），读写方便，使用灵活，常用做主存和 cache
只读存储器 ROM
顺序存取存储器 SAM
- （3）按存储介质分类
半导体存储器、磁表面存储器、光存储器

3. 主存的主要技术指标

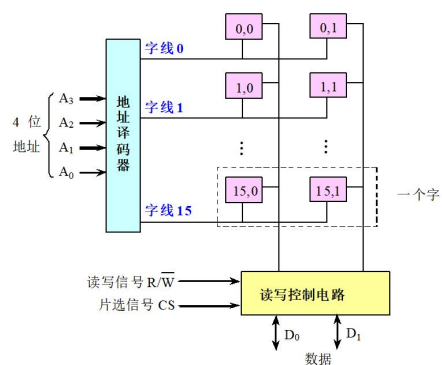
- 存储容量：一个存储器中所能存储的二进制信息的总量
- 存取速度：指访问存储器的速度
 - 存取时间 TA：指从启动一次存储器访存操作到完成该操作所需要的时间
 - 存储周期 TM：指连续两次启动存储器访问所需的最小时间间隔
 - 主存带宽 BM：存储器单位时间内所能存取的信息量
- 可靠性：在规定的时间内，存储器无故障读写的概率
- 价格

4. 随机存储器

- （1）静态随机存储器（SRAM）:利用触发器来存储二进制信息
优点：SRAM 工作速度快，稳定可靠，不需要外加刷新电路，从而简化了外电路设计
缺点：所含晶体管较多，故集成度较低，功耗较大
- （2）动态随机存储器（DRAM）:利用 MOS 晶体管的管极电容来存储二进制信息,电容上有无电荷来表示 1 或 0
优点：基本存储位元电路中所含晶体管数目少、集成度高、成本低、功耗小
缺点：它需外加刷新电路，工作速度比 SRAM 慢得多（破坏性读出且需要刷新）

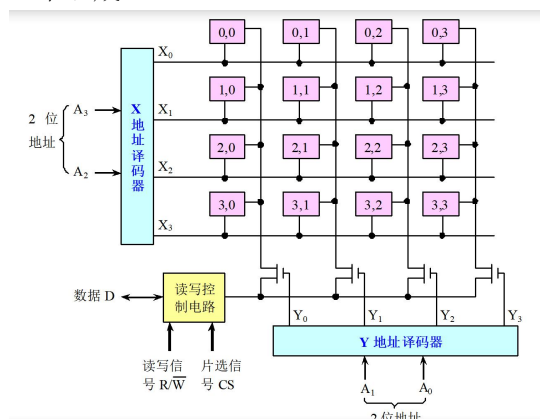
5. 芯片的内部组成方式

- 一维线性组成：（片选信号为低电平有效）

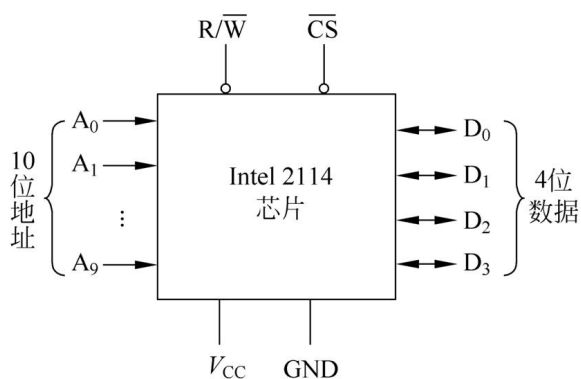


缺点：当地址位数 n 较大时，译码器的规模随之增大很多，导致电路复杂，译码时间很长，存储芯片的速度太慢

二维组成：



优点：适合用于构造大容量的存储芯片



R/w 为写允许信号（低电平有效）

CS 是片选信号（低电平有效），片选信号控制芯片是否工作，只要有效时，芯片才进行读或写操作

Vcc 是电源端，GND 为接地端

6. 动态 RAM 的刷新（也常考）

什么叫刷新：为维持 DRAM 所存信息不变，需要定时地对 DRAM 中的电容充电，以补充泄

漏掉的电荷。这个过程叫刷新

DRAM 为什么需要刷新：DRAM 是利用电容上保存的电荷来存储信息的，由于存在漏电阻，即使电源不掉电，时间长了，电容上的电荷也会慢慢泄漏掉，DRAM 内存储的信息会自动消失。

什么叫刷新周期：从上一次对整个存储器刷新结束到下一次对整个存储器刷新结束所需的时间

3 种不同的刷新方式

1.集中式刷新：在一个刷新周期内，集中一段时间连续地对全部存储单元逐行刷新一遍。

在刷新操作期间，不允许 CPU 对存储器进行正常的访问

优点：读写操作时不受刷新工作的影响，系统的存取速度比较高

缺点：在集中刷新期间必须停止读写，这一段时间称为“死区”，而且存储容量越大，死区就越长。

2. 分散式刷新：把对每行存储单元的刷新分散到每个系统存取周期内完成。此时系统存取周期被分为两部分，周期前半段时间进行正常的存储器访问，后半段时间进行刷新操作。在一个系统存取周期内刷新存储矩阵中的一行。 增加了系统的存取周期

优点：没有死区

缺点：刷新过于频繁。系统存取周期是存储芯片存取周期的两倍，降低了访问存储器的速度。

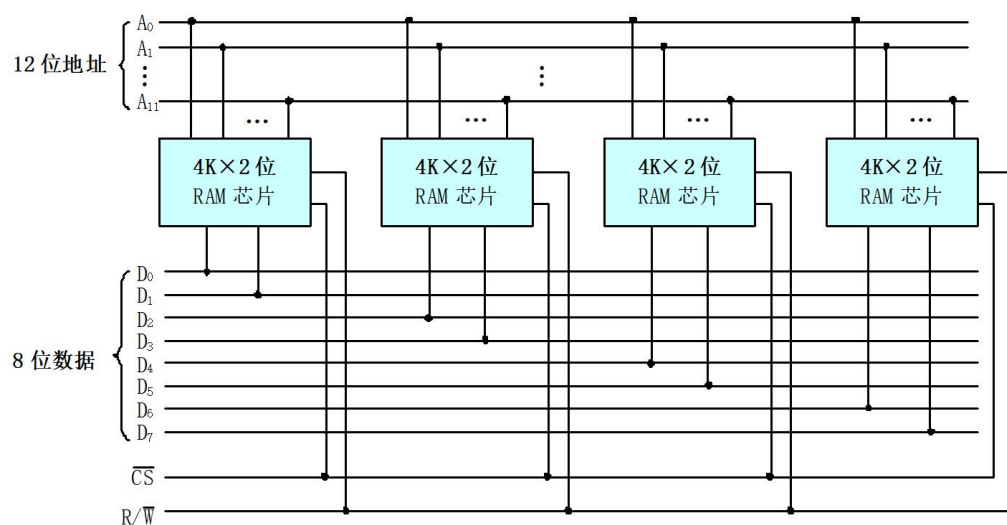
3. 异步式刷新：把刷新操作平均分配到整个最大刷新间隔内进行，相邻两行的刷新间隔为：最大刷新间隔时间÷行数

7. 只读存储器 ROM

8. 主存的设计（必考大题）

注意，每片芯片都要画 Vcc 和接地，A11 是高位位扩展

例如：用4K×2位的RAM存储芯片构成4K×8位的存储器。
用了4片RAM存储芯片。



字扩展

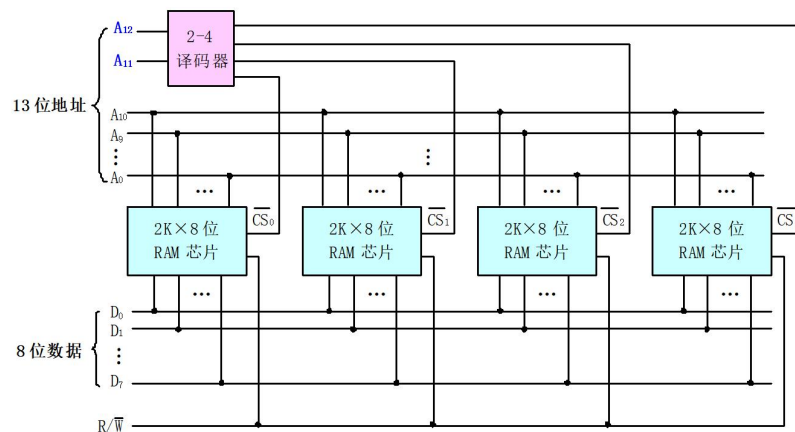
要区分高位交叉和低位交叉的区别

2-4 译码器：00-》1110，01-》1101，10-》1011，11-》0111

片选信号 CS 是低电位有效的，所以通过上面 4 个信号可以控制哪些芯片有效

(1) 高位交叉（一般字扩展都是使用高位交叉的）

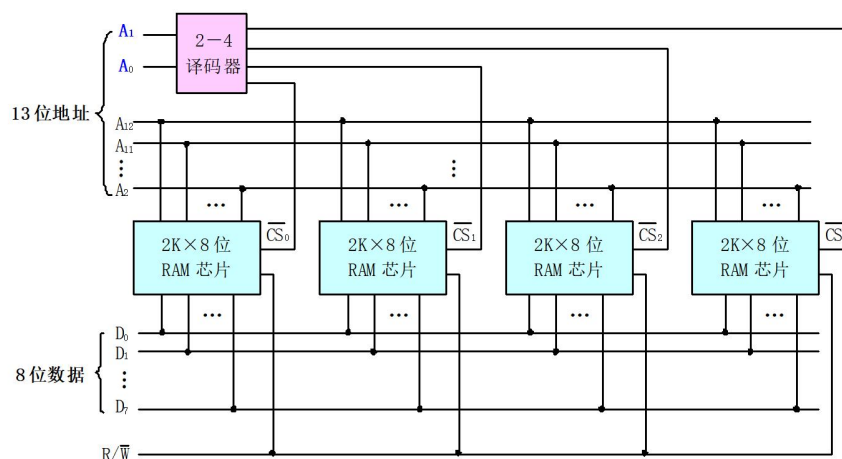
高位交叉：选择高若干位地址形成片选线。



高位交叉下，应该是第一块芯片用完了，才开始使用第二块芯片，以此类推，这意味着同一芯片的所有存储单元在存储器中的地址是连续的

(2) 低位交叉

低位交叉：选择低若干位地址形成片选线。

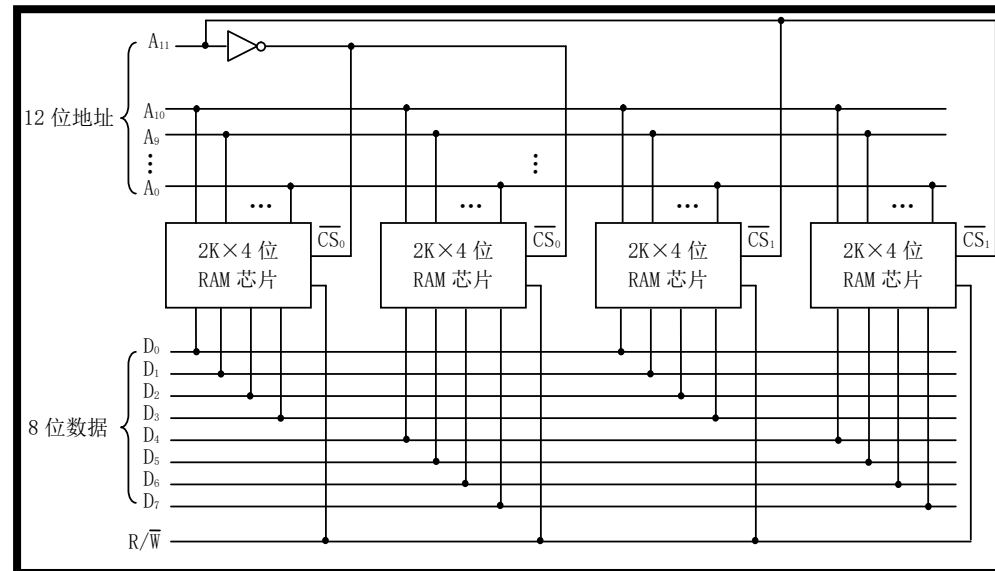


低位交叉下，四块芯片是依次轮流式有效的
这意味着同一芯片内的存储单元的存储地址是不连续的

字和位同时扩展法

常见的是组内进行位扩展，组间进行字扩展

例如：使用 $2K \times 4$ 位的存储芯片构成 $4K \times 8$ 位存储器，则需要在位和字两个方向同时扩展共两组，每组两片



课后习题

11.10 用 $16K \times 8$ 位的 DRAM 芯片构成 $64K \times 32$ 位。

$$4 \times 4 = 16$$

(1) 画出该存储器的组成框图。

(2) 设该存储器读/写周期为 $0.5\mu s$, CPU 在 $1\mu s$ 内至少要访问一次。试问采用哪种刷新方式比较合理？两次刷新的最大时间间隔是多少？对全部存储单元刷新一遍所需的实际刷新时间是多少？

11.11 某计算机系统具有 6 位的地址总线和 8 位的数据总线。它有 32 字节的 ROM，起始地址为 $00H$ ，由 32×2 的芯片构成；有 32 字节的 RAM，起始地址为 $20H$ ，由 16×4 的芯片构成；试画出存储器子系统的组成框图。

$$32 \times 8$$

$$32 \times 8$$

这题有几个注意点：32 字节不用管， $20H$ 和 $00H$ 之间恰好就是 32 字节，也就是中间就是 ROM；注意 ROM 的起始地址是 $00H$ ，也就是答案应该要让 ROM 芯片组先生效；注意芯片的 CS 信号是低电平有效的，当然也可以用高电平有效的思路去解决（感觉比较简单，但这么做的话就要让生效的顺序反过来），最后在接入芯片的 CS 端之前接个取反部件即可。